

Politechnika Warszawska
Wydział Elektroniki i Technik Informacyjnych

SYKOM

PROJEKT

Autorka:
NATALIA ŚLEPOWROŃSKA

Warszawa 2025L

Spis treści

1. Zadanie projektowe	2
2. Założenia projektu	3
2.1. Rejestry	3
3. Logika liczenia CRC16DECTR	3
4. Przesunięcia bitowe	4
5. Testbench	5
6. Aplikacja testowa	10
7. Wnioski	12

1. Zadanie projektowe

Osoba dla której przygotowano ten dokument Slepowska Natalia

Zadanie projektowe: Należy realizować korzystając z tzw. wirtualnych komputerów rezerwowanych za pomocą serwera WWW <https://resrepo.tele.pw.edu.pl>. Rezerwacje dla projektu są dokonywane w slotach maksymalnie 2h. Maszyny można rezerwować wielokrotnie lecz liczba rezerwacji dla poszczególnych etapów jest ograniczona. Projekt realizowany jest indywidualnie (grupa jedno osobowa)!

Zgodnie z opisem wprowadzającym [SYKOM_proj.pdf](#) proszę utworzyć:

- Moduł Verilog w pliku: gpioemu.v, który ma realizować operację wyznaczania sumy kontrolnej opisywanej w literaturze jako: CRC16DECTR. Suma ta ma być obliczana z treści zapisanej w bloku pamięci przechowywanym w wnętrzu modułu verilog, blok ten ma mieć wielkość 250 lokacji 8 bitowych.

Dane użytkownika mają być wprowadzone pojedynczo do tego bloku (po 8 bitów) poprzez rejestr wejściowy IN ulokowany pod adresem relatywnym: 0xCC0 w przestrzeni pamięciowej procesora. Użytkownik gdy uzna, iż zapełnił blok danymi i chciałby aby moduł wykonał obliczenie sumy kontrolnej przekazuje poprzez rejestr CTRL umiejscowiony pod adresem relatywnym 0xCD8, wartość będącą odpowiednio zaprojektowanym poleceniem, innymi poleceniami mogłyby być np.: wyczyść cały blok, ustaw pozycję zapisywania danych użytkownika w wnętrzu bloku na pozycję 0, itp. W raporcie przedstaw analizę przyjętej przez Ciebie listy wspieranych poleceń i sposób ich bitowego kodowania.

Stan modułu np.: zapełnienie bloku, zakończenie liczenia sumy kontrolnej i innych podobnych sytuacji, ma być udostępniany procesorowi poprzez rejestr STATE ulokowany pod adresem relatywnym 0xCC8 w jego przestrzeni pamięciowej.

Za każdym razem gdy moduł poprawnie obliczy sumę kontrolną, jej wynik ma być dostępny dla procesora poprzez ulokowany pod adresem relatywnym 0xCD0 rejestr RESULT.

Pamiętaj aby moduł Verilog był zapisany optymalnie (m.in.: usuwaj z jego kodu to co nie potrzebne, co nie jest zapisane w tej specyfikacji lub jest w tym kodzie poprostu nadmiarowe). W kodzie tego modułu nie stosuj żadnych zewnętrznych bibliotek, nie stosuj także żadnych postaci pętli a kod Verilog niech wykorzystuje wyłącznie tzw. automaty stanów. Stopień spełnienia tych wymagań będzie oceniany. Nie zapomnij przetestować pliku Verilog odpowiednim (zgodnie z zasadami przekazanymi na laboratorium nr. 2 plikami testującymi które umieść także na GIT a ich działanie przedstaw w raporcie.

- Dla systemu Linux utwórz odpowiedni moduł jądra który udostępni użytkownikowi dostęp do rejestrów modułu Verilog za pomocą systemu plików SYSFS tak aby rejestry te dostępne były w tym systemie plików jako: IN: dsslna, CTRL: dtslna, STATE: dcslna, RESULT: drslna. Przyjmij, że dane przekazywane poprzez te pliki są w tekstowym formacie HEX. Pamiętaj o należytej obsłudze błędnych danych przekazywanych podczas współpracy z użytkownikiem czy błędnych sytuacji pojawiających się podczas pracy tak utworzonego systemu.
- Aby łatwiej testować i pokazać poprawność działania zbudowanego systemu - w szczególności ukazać zależności czasowe - proszę utworzyć w języku C aplikację przygotowaną do pracy w zbudowanym systemie Linux. Aplikacja ta w postaci binarnej ma być wbudowana w docelowy główny system plików (czyli w plik: rootfs.ext2) i ma ona po uruchomieniu przez użytkownika automatycznie demonstrować działanie systemu w różnych scenariuszach testowych. Pamiętaj, że podejście w testowaniu z wykorzystaniem systemowych poleceń cat i echo choć może zastąpić konieczność tworzenia powyższej aplikacji w C jest jednak oceniane niżej.

Zadbaj aby na GIT znalazły się wszystkie utworzone przez Ciebie artefakty, jednakże nie przenoś do tego repozytorium zbędnych elementów, m.in. produktów kompilacji, czyli produktów tworzonych przez udostępnione narzędzia. Spełnienie także tych wymagań będzie oceniane.

Rysunek 1: Zadanie projektowe

2. Założenia projektu

W zadaniu projektowym zostały zdefiniowane cztery adresy:

- 0xCC0 - pod tym adresem odbywa się zapis danych do rejestru IN
- 0xCD8 - pod tym adresem odbywa się zapis danych do rejestru CTRL
- 0xCC8 - pod tym adresem odbywa się odczyt danych z rejestru STATE
- 0xCD0 - pod tym adresem odbywa się odczyt danych z rejestru RESULT

W trakcie testów modułu verilog dodałam jeszcze dwa złe adresy, w celu testu neutralności oraz podtrzymania:

- 0xCC4
- 0xCCC

2.1. Rejestry

Rejestr IN zdefiniowałam jako 8-bitowy rejestr z przesunięciem 0 bit. Wartość rejestru IN jest przekazywana do pamięci na kolejne pozycje. Pamięć posiada 250 lokacji po 8 bit.

Rejestr CTR zdefiniowałam jako 2-bitowy rejestr z przesunięciem 8 bit. Może on przyjmować trzy wartości:

- 0x0 - komenda czyszczenia pamięci
- 0x1 - komenda ustawienia indeksu na pozycję 0 w pamięci
- 0x2 - komenda startu liczenia CRC

Rejestr STATE zdefiniowałam jako 2-bitowy rejestr z przesunięciem 10 bit. Może on przyjmować cztery wartości:

- 0x0 - pusta pamięć, oczekiwanie na dane
- 0x1 - wprowadzono dane, oczekiwanie na komendę startu liczenia
- 0x2 - moduł liczy CRC
- 0x3 - CRC zostało policzone

Rejestr REULT zdefiniowałam jako 16-bitowy rejestr z przesunięciem 0 bit. Zwraca on wynik liczenia CRC. Dodatkowo zadeklarowałam dwa rejestry sterujące:

- `crc_count_sig` - steruje liczeniem CRC
- `zero_sig` - steruje zerowaniem pamięci

3. Logika liczenia CRC16DECTR

Gdy `crc_count_sig` przyjmuje wartość 0x1, rozpoczyna się liczenie CRC. Program przetwarza każdy bajt w pamięci (memory), analizując go bit po bicie.

Dla pierwszego bitu (0), bieżący bajt z pamięci jest przesuwany o 8 bitów w lewo. Następnie wykonywana jest operacja XOR bajtu z aktualną wartością rejestru `crc`, który początkowo wynosi 0x0000.

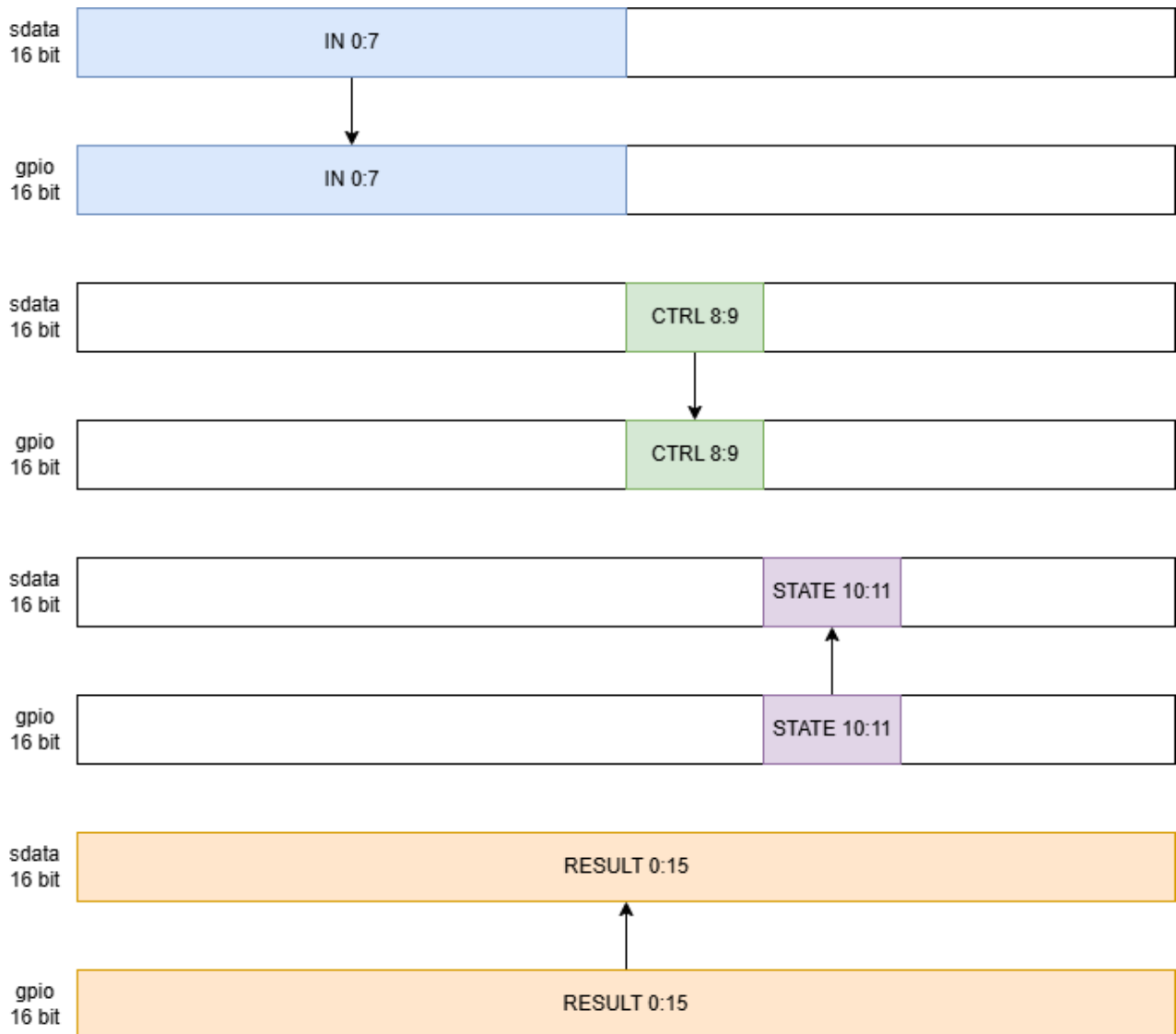
Następnie przez kolejne osiem kroków, rejestr `crc` jest przesuwany o 1 bit. Po każdym przesunięciu, jeśli najstarszy bit (15) posiada wartość 1, rejestr `crc` jest poddawany operacji XOR z wielomianem 0x0589.

Na koniec rejestr `crc` jest poddawany operacji XOR z wartością 0x0001 i przekazywany do rejestru RESULT.

Poprawność liczenia CRC16DECTR weryfikowałam w wykorzystaniem kalkulatora online: [link](#).

4. Przesunięcia bitowe

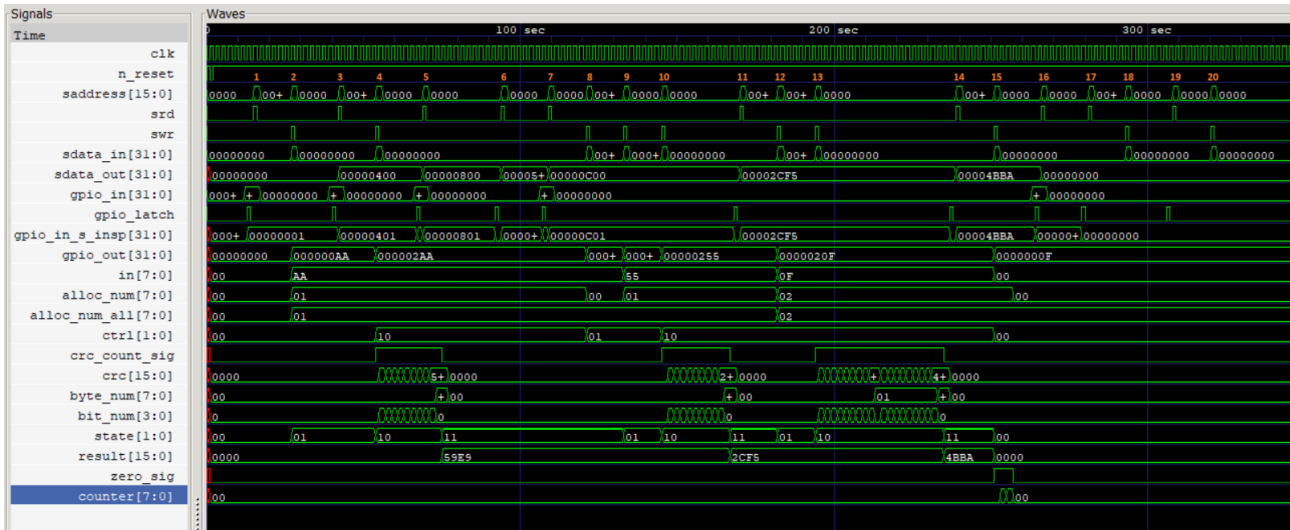
Przesunięcia bitowe dla odpowiednich rejestrów:



Rysunek 2: Przesunięcia

5. Testbench

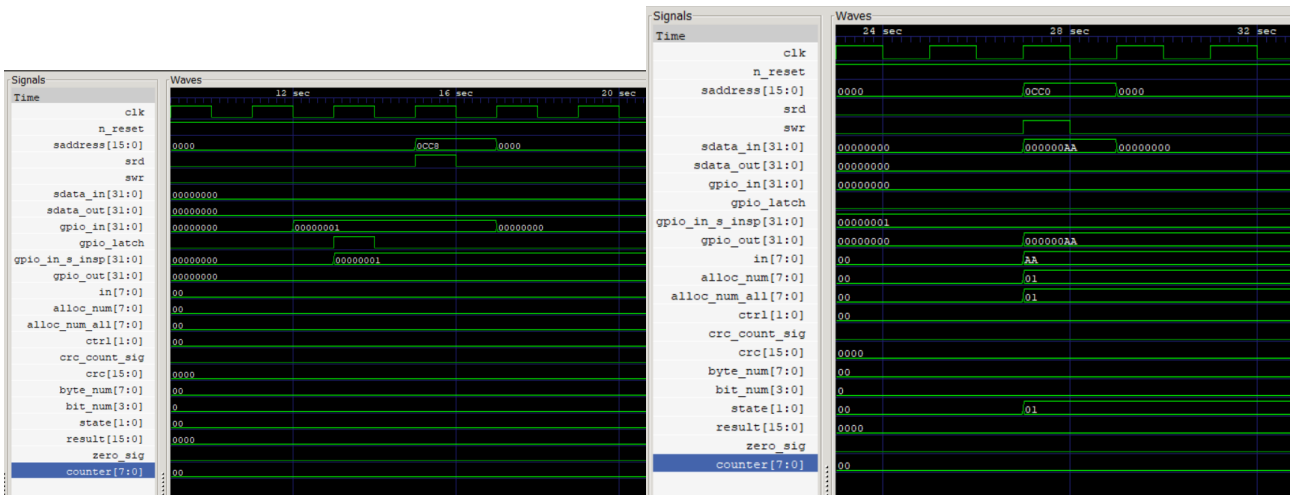
Dla mojego modułu verilog przygotowałam testbench do weryfikacji poprawności jego działania. Składa się on z 20 testów:



Rysunek 3: Testbench

Test 1 (odczyt STATE): Podaję dane do rejestru gpio_in i wykonuję test odczytu dla adresu 0xCC8. Rejestr sdata_out nie zmienił swojej wartości - ponieważ nastąpiło przekazanie wartości rejestru STATE który obecnie przyjmuje wartość 0x0.

Test 2 (zapis IN): Wykonuję test zapisu wartości 0xAA pod adres 0xCC0. Wartość ta jest przekazana do rejestrów gpio_out oraz IN. Rejestr alloc_num i alloc_num_all przyjmują wartość 0x1 co oznacza odpowiednio pozycję 1 w pamięci i liczbę wszystkich zapisów równą 1. Rejestr STATE także przyjął wartość 0x1.

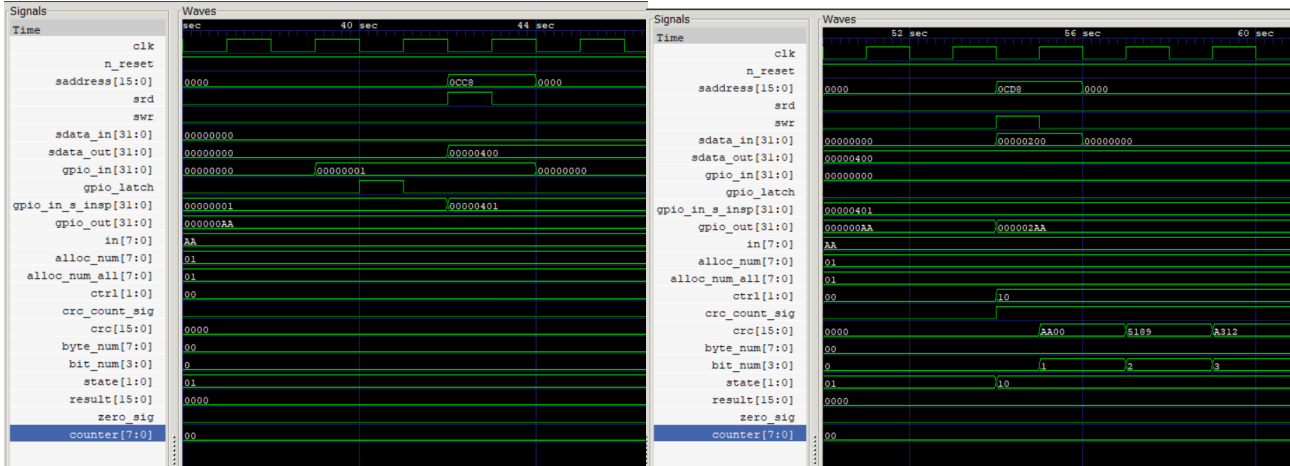


((a)) Test 1

((b)) Test 2

Test 3 (odczyt STATE): Podaję dane do rejestru gpio_in i wykonuję test odczytu dla adresu 0xCC8. Rejestr sdata_out zmienia swoją wartość na 0x400 - ponieważ nastąpiło przekazanie wartości rejestru STATE, przyjmuje on obecnie wartość 0x1.

Test 4 (zapis CTRL): Wykonuję test zapisu wartości 0x200 pod adres 0xCD8. Wartość ta zostaje przekazana do rejestru CTRL. Rejestr STATE przyjmuje wartość 0x2.

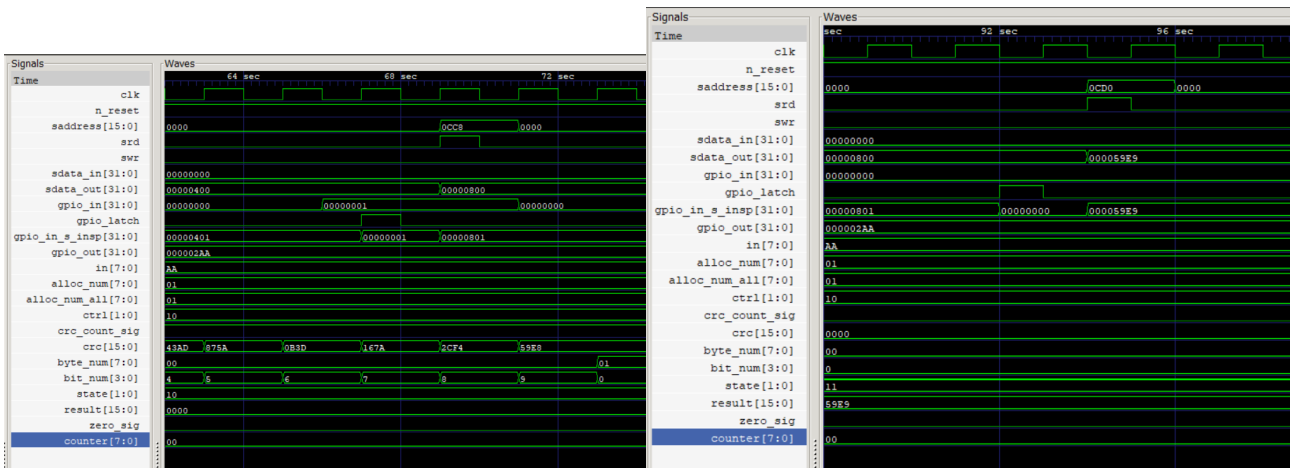


((a)) Test 3

((b)) Test 4

Test 5 (odczyt STATE): Podaję dane do rejestru gpio_in i wykonuję test odczytu dla adresu 0xCC8. Rejestr sdata_out zmienia swoją wartość na 0x800 - ponieważ następuje przekazanie wartości rejestru STATE, przyjmuje on obecnie wartość 0x2.

Test 6 (odczyt RESULT): Podaję dane do rejestru gpio_in i wykonuję test odczytu dla adresu 0xCD0. Rejestr sdata_out zmienia swoją wartość na 0x59E9 - ponieważ następuje przekazanie wartości rejestru RESULT.

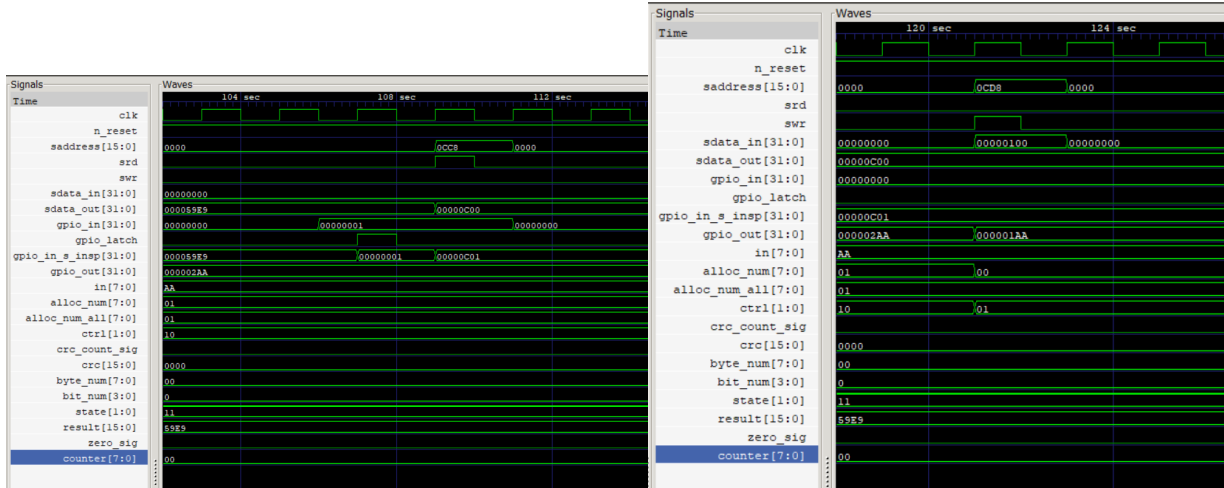


((a)) Test 5

((b)) Test 6

Test 7 (odczyt STATE): Podaję dane do rejestru gpio_in i wykonuję test odczytu dla adresu 0xCC8. Rejestr sdata_out zmienia swoją wartość na 0xC00 - ponieważ następuje przekazanie wartości rejestru STATE, przyjmuje on obecnie wartość 0x3.

Test 8 (zapis CTRL): Wykonuję test zapisu wartości 0x100 pod adres 0xCD8. Wartość ta zostaje przekazana do rejestru CTRL. Następuje zmiana wartości rejestru alloc_num.

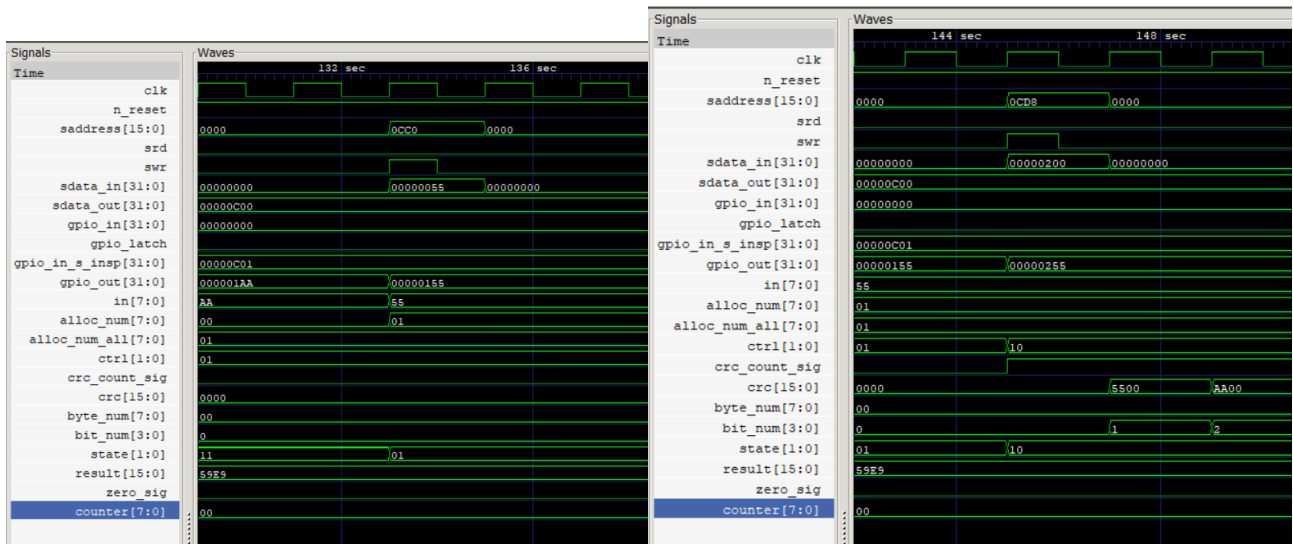


((a)) Test 7

((b)) Test 8

Test 9 (zapis IN): Wykonuję test zapisu wartości 0x55 pod adres 0xCC0. Wartość ta zostaje przekazana do rejestru IN. Rejestr STATE przyjmuje wartość 0x1.

Test 10 (zapis CTRL): Wykonuję test zapisu wartości 0x200 pod adres 0xCD8. Wartość ta zostaje przekazana do rejestru CTRL. Rejestr STATE przyjmuje wartość 0x2.

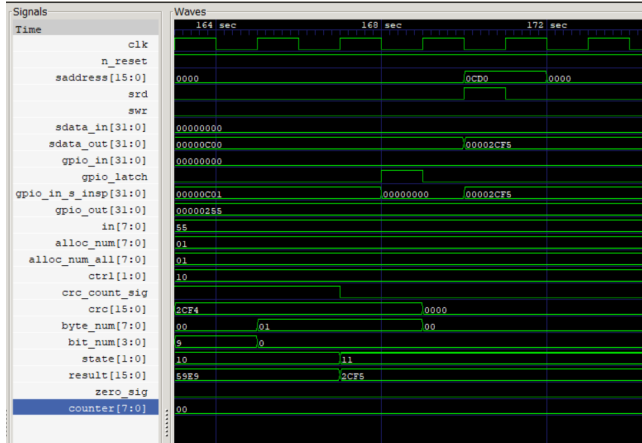


((a)) Test 9

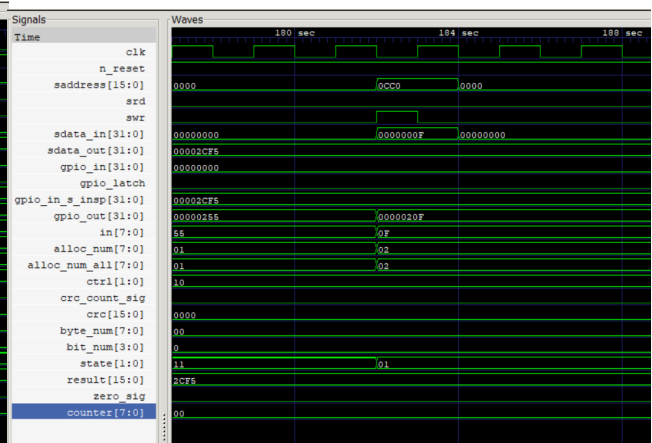
((b)) Test 10

Test 11 (odczyt RESULT): Podaję dane do rejestru gpio_in i wykonuję test odczytu dla adresu 0xCDO. Rejestr sdata_out zmienia swoją wartość na 0x2CF5 - ponieważ następuje przekazanie wartości rejestru RESULT.

Test 12 (zapis IN): Wykonuję test zapisu wartości 0xF pod adres 0xCC0. Wartość ta zostaje przekazana do rejestru IN. Rejestr STATE przyjmuje wartość 0x1.



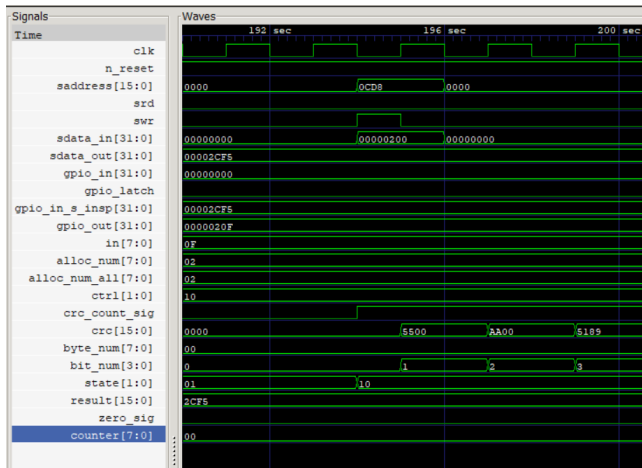
((a)) Test 11



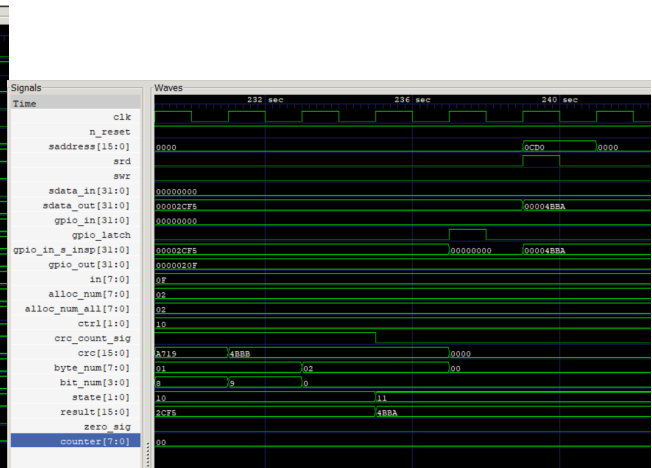
((b)) Test 12

Test 13 (zapis CTRL): Wykonuję test zapisu wartości 0x200 pod adres 0xCD8. Wartość ta zostaje przekazana do rejestru CTRL. Rejestr STATE przyjmuje wartość 0x2.

Test 14 (odczyt RESULT): Podaję dane do rejestru gpio_in i wykonuję test odczytu dla adresu 0xCDO. Rejestr sdata_out zmienia swoją wartość na 0x4BBA - ponieważ następuje przekazanie wartości rejestru RESULT.



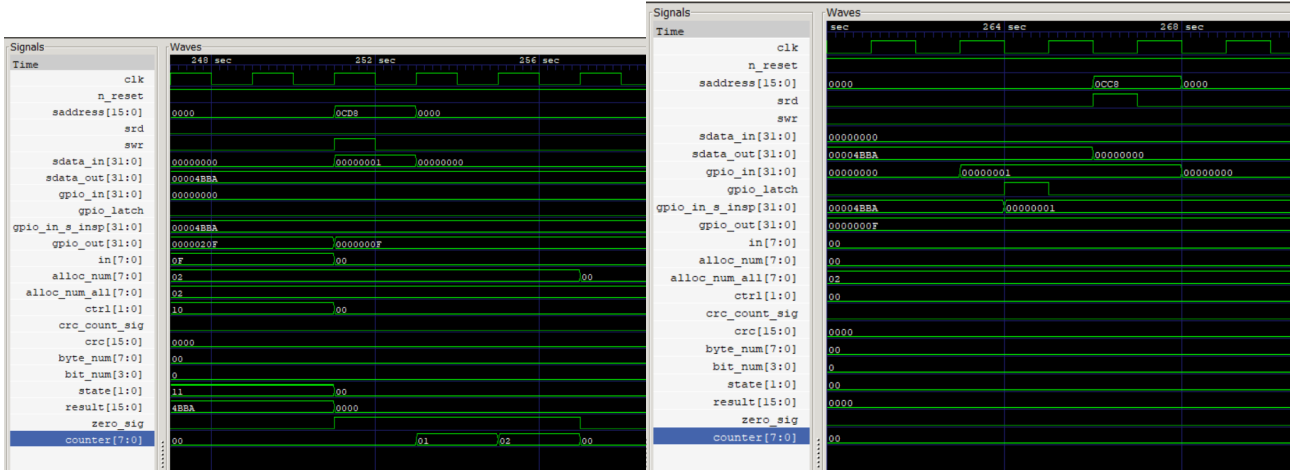
((a)) Test 13



((b)) Test 14

Test 15 (zapis CTRL): Wykonuję test zapisu wartości 0x0 pod adres 0xCD8. Wartość ta zostaje przekazana do rejestru CTRL.

Test 16 (odczyt STATE): Podaję dane do rejestru gpio_in i wykonuję test odczytu dla adresu 0xCC8. Rejestr sdata_out zmienia swoją wartość na 0x0 - ponieważ następuje przekazanie wartości rejestru STATE, przyjmuje on obecnie wartość 0x0.

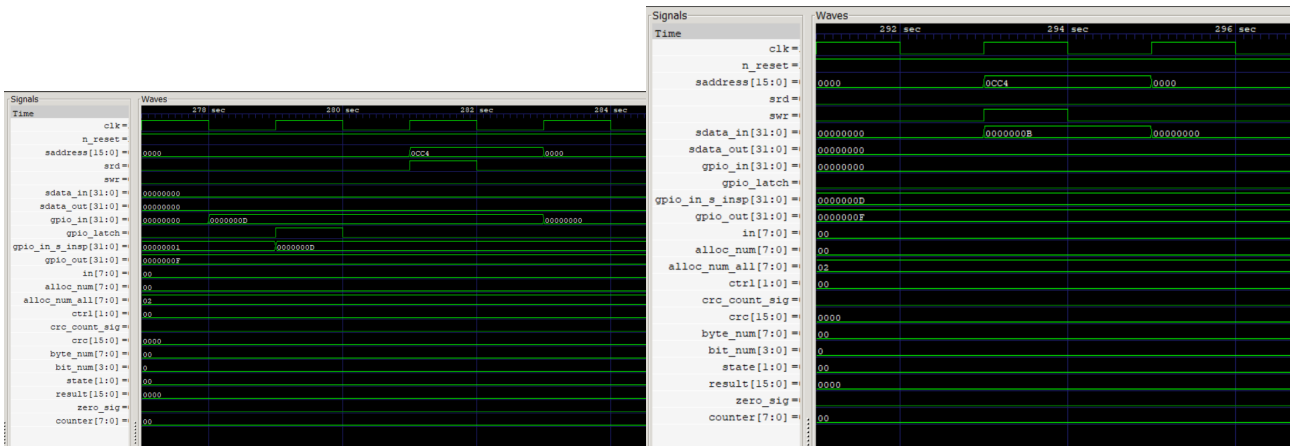


((a)) Test 15

((b)) Test 16

Test 17 (neutralność): Podaję dane do rejestru gpio_in i wykonuję test odczytu dla adresu 0xCC4. Rejestr sdata_out nie zmienił swojej wartości - test neutralności przebiegł pomyślnie.

Test 18 (podtrzymanie): Wykonuję test zapisu wartości 0xB pod adres 0xCC4. Wartość ta nie zostaje przekazana do rejestru gpio_out - test podtrzymania przebiegł pomyślnie.

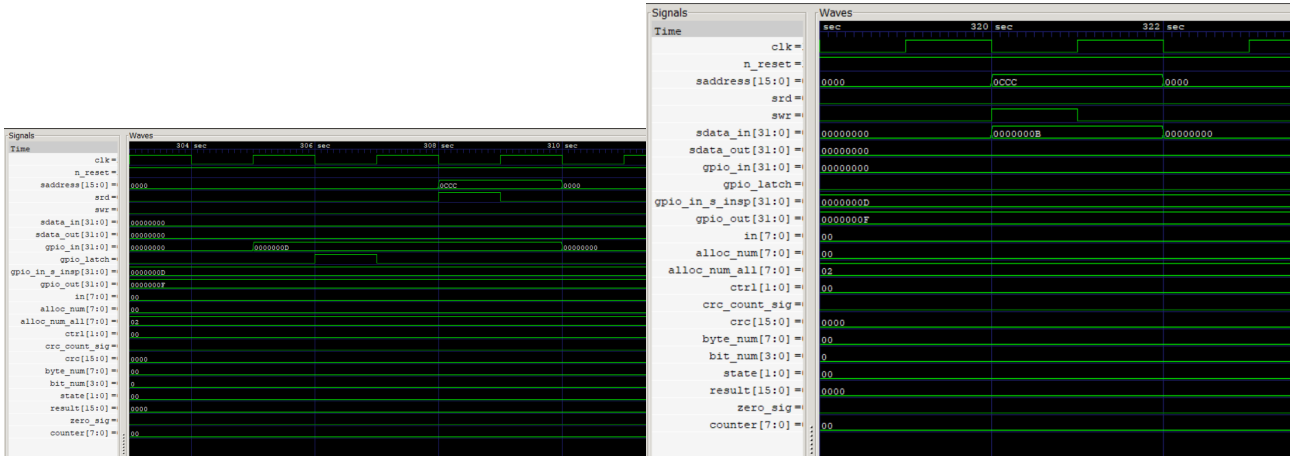


((a)) Test 17

((b)) Test 18

Test 19 (neutralność): Podaję dane do rejestru gpio_in i wykonuję test odczytu dla adresu 0xCCC. Rejestr sdata_out nie zmienił swojej wartości - test neutralności przebiegł pomyślnie.

Test 20 (podtrzymanie): Wykonuję test zapisu wartości 0xB pod adres 0xCCC. Wartość ta nie zostaje przekazana do rejestru gpio_out - test podtrzymania przebiegł pomyślnie.



((a)) Test 19

((b)) Test 20

Wszystkie testy przebiegły pomyślnie zgodnie z moimi założeniami.

6. Aplikacja testowa

Poniżej przedstawiam wynik działania mojej aplikacji testowej:

```
# modprobe kernel_module[ 15.757546] random: fast init done

[ 15.813515] Init my module.
# ./main
CRC16DECTR test 1
Current state 0 - waiting for input
Writing 0xaa input value
Current state 1 - ready for counting
Writing command 2 - start counting
Current state 2 - counting
Result: 0x59e9
Current state 3 - counting finished
CRC16DECTR test 2
Current state 3 - counting finished
Writing command 1 - back to position 0 in memory
Current state 3 - counting finished
Writing 0x55 input value
Current state 1 - ready for counting
Writing command 2 - start counting
Current state 2 - counting
Result: 0x2cf5
Current state 3 - counting finished
```

Rysunek 14: Wynik działania programu zapisanego w main.c

```

CRC16DECTR test 3
Current state 3 - counting finished
Writing 0xf input value
Current state 1 - ready for counting
Writing command 2 - start counting
Current state 2 - counting
Result: 0x4bba
Current state 3 - counting finished
CRC16DECTR test 4
Writing 0xaa input value
Writing 0xbc input value
Writing 0x8 input value
Writing 0xdd input value
Writing command 2 - start counting
Result: 0xa8d1
Czyszczenie pamieci
Writing command 0- memory cleaning
Current state 0 - waiting for input

```

Rysunek 15: Wynik działania programu zapisanego w main.c

Zrzuty ekranu kalkulatora CTR16DECTR:

AA

Input: ☐ ASCII ☒ HEX Output: ☒ HEX ☐ DEC ☐ OCT ☐ BIN ☐ Show processed data (HEX)

[Back to all algos](#) [CRC-16/DECT-R](#)

[👉 My friend's Telegram channel with cute content for every day.](#)

CRC-16/DECT-R	Result	Check	Poly	Init	RefIn	RefOut	XorOut
CRC-16/DECT-R R-CRC-16	0x59E9	0x007E	0x0589	0x0000	false	false	0x0001

Rysunek 16: CTR16DECTR test 1

55

Input: ☐ ASCII ☒ HEX Output: ☒ HEX ☐ DEC ☐ OCT ☐ BIN ☐ Show processed data (HEX)

[Back to all algos](#) [CRC-16/DECT-R](#)

[👉 My friend's Telegram channel with cute content for every day.](#)

CRC-16/DECT-R	Result	Check	Poly	Init	RefIn	RefOut	XorOut
CRC-16/DECT-R R-CRC-16	0x2CF5	0x007E	0x0589	0x0000	false	false	0x0001

Rysunek 17: CTR16DECTR test 2

550F

Input: ☐ ASCII ☒ HEX Output: ☒ HEX ☐ DEC ☐ OCT ☐ BIN ☐ Show processed data (HEX)

[Back to all algos](#) [CRC-16/DECT-R](#)

👉 My friend's Telegram channel with cute content for every day.

CRC-16/DECT-R	Result	Check	Poly	Init	RefIn	RefOut	XorOut
CRC-16/DECT-R R-CRC-16	0x4BBA	0x007E	0x0589	0x0000	false	false	0x0001

Rysunek 18: CTR16DECTR test 3

550FAABC08DD

Input: ☐ ASCII ☒ HEX Output: ☒ HEX ☐ DEC ☐ OCT ☐ BIN ☐ Show processed data (HEX)

[Back to all algos](#) [CRC-16/DECT-R](#)

👉 My friend's Telegram channel with cute content for every day.

CRC-16/DECT-R	Result	Check	Poly	Init	RefIn	RefOut	XorOut
CRC-16/DECT-R R-CRC-16	0xA8D1	0x007E	0x0589	0x0000	false	false	0x0001

Rysunek 19: CTR16DECTR test 4

Dane	Wynik testbench	Wynik main.c	Wynik kalkulator
0xAA	0x59E9	0x59E9	0x59E9
0x55	0x2CF5	0x2CF5	0x2CF5
0x550F	0x4BBA	0x4BBA	0x4BBA
0x550FAABC08DD	-	0xA8D1	0xA8D1

Tabela 1: Wyniki liczenia CRC16DECTR

7. Wnioski

Udało mi się wykonać zadanie projektowe zgodnie z moimi założeniami. Licznik CRC16DECTR działa poprawnie w zdefiniowanym środowisku. Zwiększyłam swoją wiedzę z programowaniem w językach C i Verilog oraz dotyczącą systemu Linux.